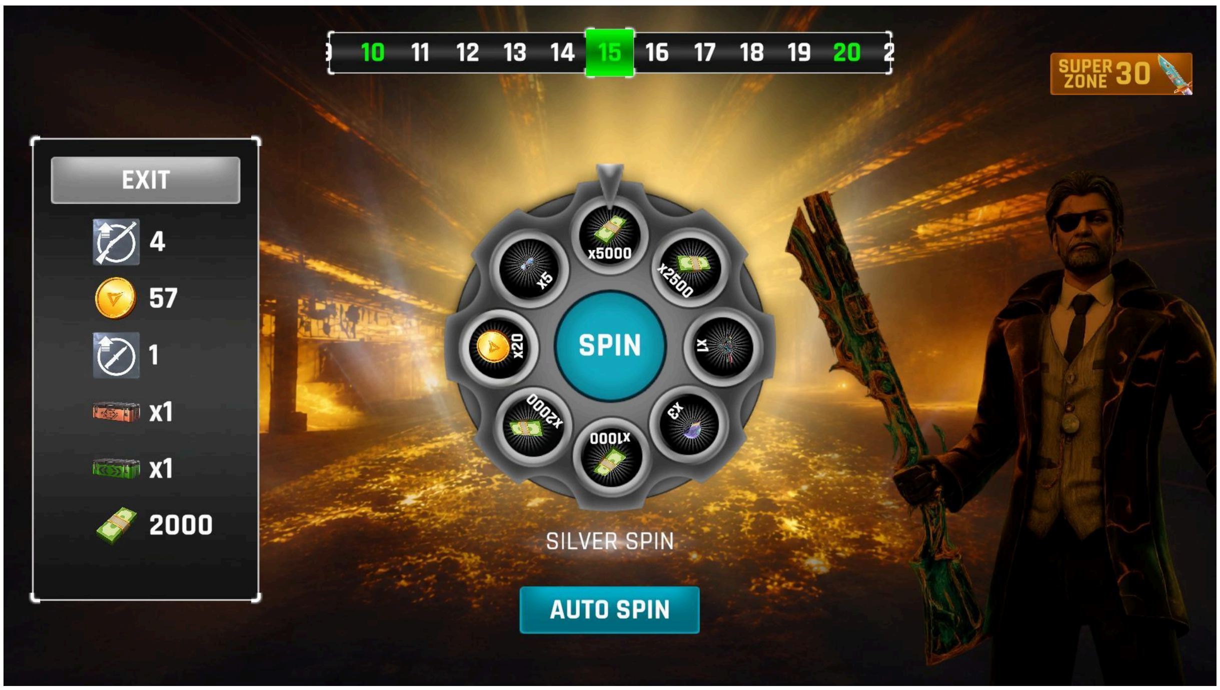
WHEEL OF FORTUNE

Developer Technical Documentation

Vertigo Games - Game Developer Case

This document created by okarosa - Kerem Hünsoy



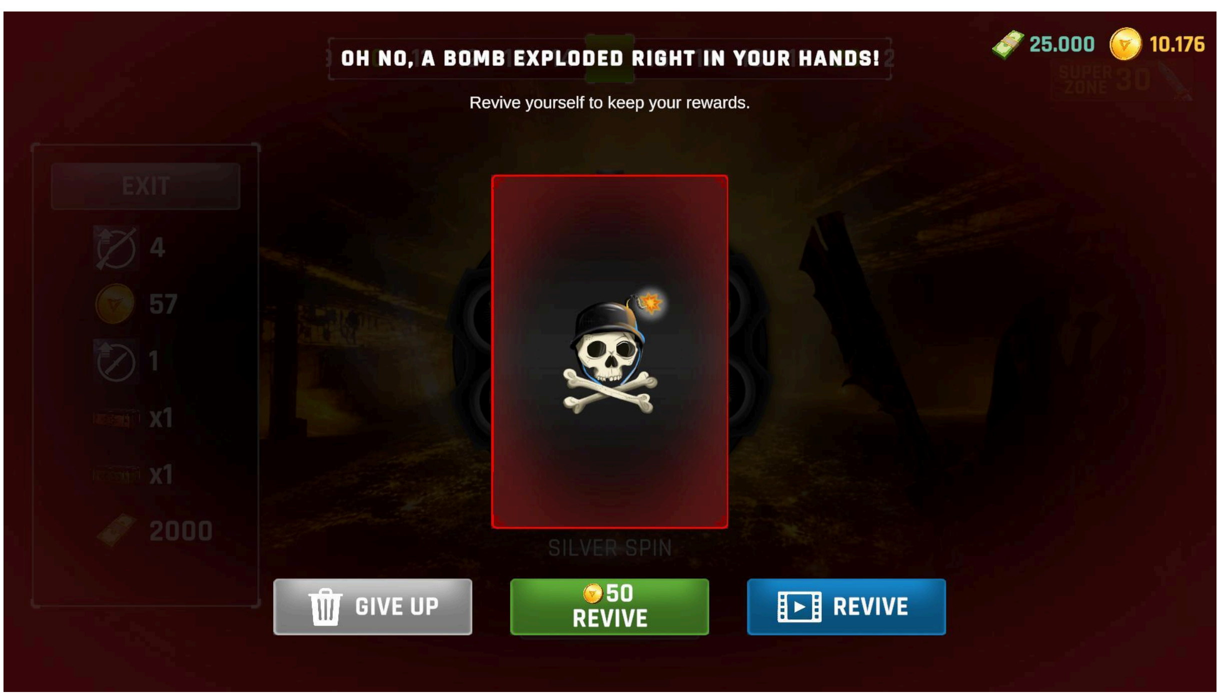


Table of Contents

1. Project Overview

Wheel of Fortune is a mobile-first Unity game built as a case study for Vertigo Games. The design closely mirrors the Card Game mode visual language of Critical Strike, using matching fonts, color palettes, and animation timing. All asset content comes from the provided demo content pack.

The player spins a wheel zone by zone, collecting rewards. Each zone the wheel contains reward slices and one bomb. Landing on the bomb ends the session and all collected rewards are lost - unless the player revives. Safe and Super zones provide guaranteed bomb-free spins.

Zone Types

Zone	Type	Wheel	Bomb
1–4, 6–9 …	Normal	Bronze	✔ Yes
5, 10, 15 …	Safe	Silver	✘ No
30	Super	Gold	✘ No
60	Final (Win)	Gold	✘ No

The player may cash out and keep all earnings at any Safe or Super zone before spinning. The game ends (victory) when zone 60 is completed. There is no limit to revives as long as the player has enough coins.

2. Architecture

The project uses a Service Locator pattern (GameServices) as the central dependency injection point. All managers register themselves on Awake and retrieve dependencies through typed Get<T>() calls - no direct cross-references between managers.

High-Level Graph

```
GameServices (service locator)
├── GameManager        - state machine
├── AudioManager       - SFX + music
├── RewardCollector    - session + persistence
├── DemoSettings       - dev tooling
├── ZoneProgressBar    - zone UI
├── WheelController    - spin logic
└── UIManager (facade)
    ├── HUDHandler
    ├── PanelController
    └── RewardFeedHandler
```

Managers

Class	Responsibility
`GameServices`	Central service locator - Register<T>() / Get<T>() / TryGet<T>()
`GameManager`	State machine: Idle → Spinning → Result → Bomb → RewardComplete
`AudioManager`	Dictionary-keyed SFX bank, per-wheel-tier looping background music
`RewardCollector`	Session reward tracking, PlayerPrefs persistence on cash-out
`DemoSettings`	Dev tooling - configurable starting values, PlayerPrefs reset
`ZoneProgressBar`	Scrollable zone indicator, color-coded tier fills, animated transitions

Controllers

Class	Responsibility
`WheelController`	Spin physics, slice randomization, idle rotation, auto-spin, result flow

UI Handlers

Class	Responsibility
`UIManager`	Facade - holds serialized UI references, delegates to handlers, button wiring
`HUDHandler`	Zone info, coin display, exit / auto-spin button state, character idle animation
`PanelController`	Bomb / Game Over / Reward Complete / Buy Coin panel animations and flow
`RewardFeedHandler`	Reward list item creation, coin fly and collect icon effects, smooth scroll

ScriptableObjects

Class	Responsibility
`WheelSliceData`	Icon, display name, reward amount, bomb flag per slice
`WheelConfigData`	Reward pool array, wheel visuals, hasBomb flag, background music clip
`BuyCoinConfig`	Three-tier coin purchase configuration

3. Editor Tools

All custom editor tooling lives under the Unity menu bar at: Tools → Wheel of Fortune. There are four items available:

- Player Settings
- Slice Data Creator
- View Slice Datas
- Setup Scene Hierarchy

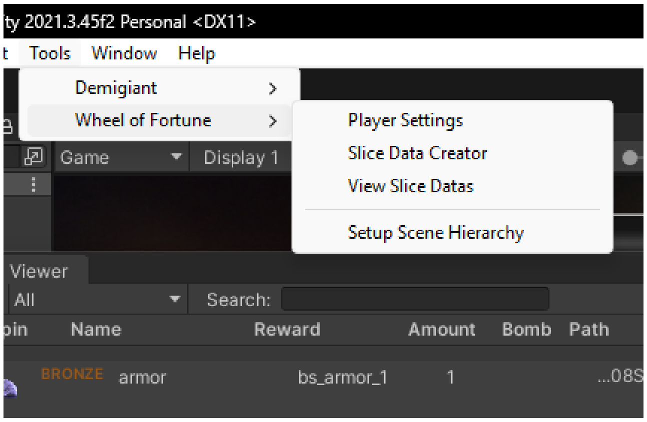


Figure 1 - Tools → Wheel of Fortune menu in Unity 2021.3.45f2

Player Settings

Accessible via Tools → Wheel of Fortune → Player Settings. Opens a dedicated editor window (DemoSettingsEditor) that lets QA or developers configure starting currency values without touching the code.

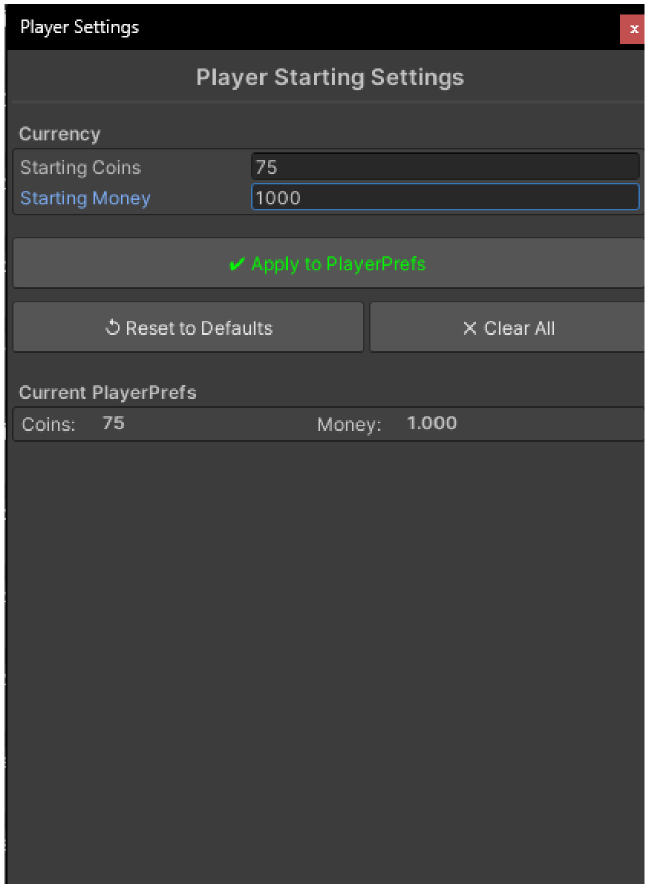


Figure 2 - Player Starting Settings window. Values are applied directly to PlayerPrefs.

The window exposes:

- Starting Coins - default coin balance before the first session
- Starting Money - default money balance
- Apply to PlayerPrefs - writes current field values to PlayerPrefs immediately
- Reset to Defaults - restores factory default values
- Clear All - deletes all PlayerPrefs keys (full wipe)
- Current PlayerPrefs panel - live read-back of stored values

Slice Data Viewer

Accessible via Tools → Wheel of Fortune → View Slice Datas. Displays all WheelSliceData ScriptableObjects in the project in a searchable, sortable table. Each row shows the icon, tier, name, reward key, amount, bomb flag, and asset path.

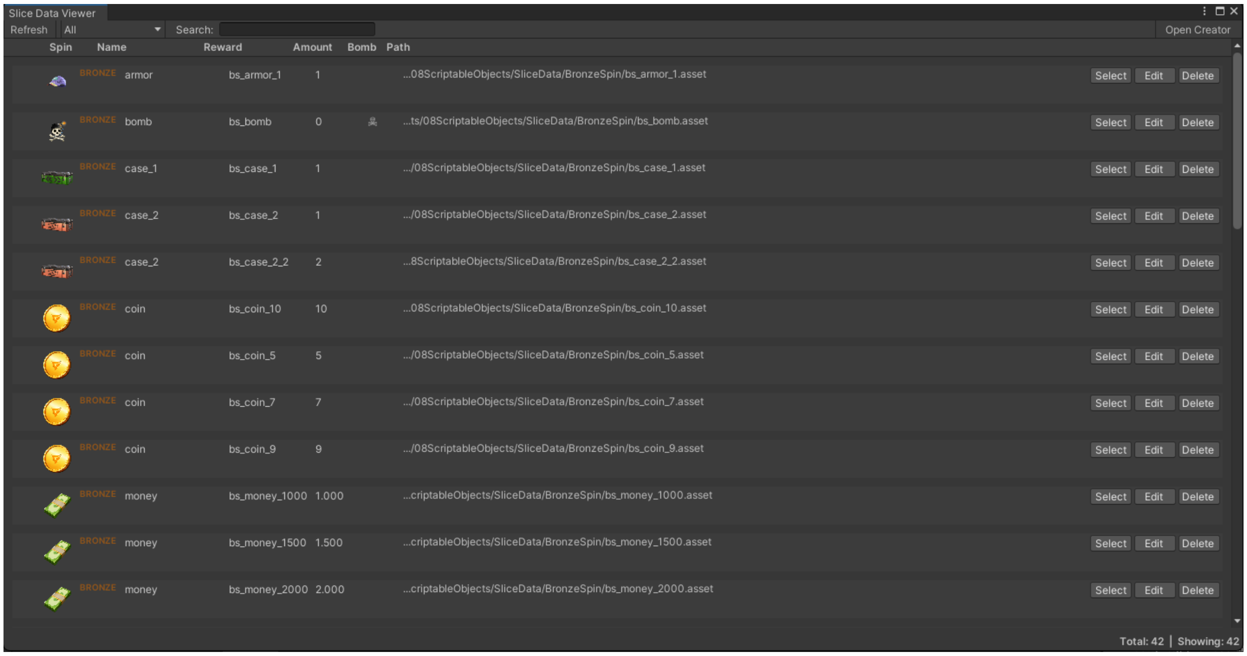


Figure 3 - Slice Data Viewer showing 42 Bronze spin entries. Bomb column marks the bomb slice.

Column breakdown:

- Spin - tier badge (BRONZE / SILVER / GOLD) + reward icon
- Name - display name shown in-game
- Reward - internal reward key used to look up values at runtime
- Amount - base reward amount for this slice
- Bomb - bomb icon present only when IsBomb = true
- Path - asset path in the project for quick navigation

Each row has Select, Edit, and Delete buttons. Select pings the asset in the Project window. Total and Showing counts are shown in the bottom-right corner. The search bar filters by name in real time.

Slice Data Creator

Opens a guided creation wizard for new WheelSliceData assets. Fields match the WheelSliceData ScriptableObject: icon sprite, display name, reward key, amount, bomb flag, and output path. Saves the asset to the configured SliceData folder.

Setup Scene Hierarchy

One-click scene setup tool. Creates and names all required GameObjects in the correct hierarchy with the right components pre-attached. Intended for first-time scene setup on a fresh project - do not run on an already-configured scene.

4. Technical Details

Code Quality

- SOLID principles throughout - each manager has a single clear responsibility
- WheelConfigData and WheelSliceData are open for extension without modifying existing classes (Open/Closed)
- No coroutine-based animation anywhere - all motion handled exclusively through DOTween
- All button references resolved automatically via OnValidate - no Unity Editor onClick or event references needed

UI & Layout

- Canvas Scaler set to Expand mode - anchors and pivots verified at 20:9, 16:9, and 4:3
- TextMeshPro used for all text elements
- Sliced Sprites on all Image components - no stretching at any resolution
- RaycastTarget and Maskable disabled on all non-interactive images
- Animator components placed on dedicated child transforms, never on root
- Dynamic UI elements follow _value suffix convention (e.g. ui_text_coin_value)
- Hierarchy naming follows root-to-specific pattern (e.g. ui_image_spin_silver)

Data & Persistence

- RewardCollector holds session state - flushed to PlayerPrefs on cash-out, discarded on give-up
- WheelSliceData and WheelConfigData ScriptableObjects allow full wheel content editing from the Inspector without a code change
- DemoSettingsEditor provides a custom inspector with Clear and Reset tools for testing

Rendering

- Sprite Atlas for draw call batching
- Instance materials used for per-object visual effects (bomb flash, wheel fade)

5. Animations

All animation is implemented with DOTween - no coroutines, no Unity Animator state machines for gameplay motion.

Effect	Implementation
Wheel Spin	OutCubic ease, 5–9 extra full rotations, overshoot and bounce-back on result

Effect	Implementation
Collect Fly	Icon travels from wheel center to reward list entry with rotation
Coin Revive	5 coins spawn at button, scatter randomly, converge to coin icon with progressive shrink; target scales to 1.3× on arrival
Reward Stack	New items scale and fade in; existing items pulse and count up with per-increment tick sound
Bomb	Screen shake, bomb icon scale and fade
Character Idle	Scale breathing 1.0 ↔ 1.03 on a 2-second loop
Zone Progress	Fill bar advances, text crossfades, content scrolls on zone change

6. Audio

AudioManager maintains a dictionary-keyed SFX bank and separate looping music channels. Background music switches automatically when the wheel tier changes zone (Bronze → Silver → Gold).

SFX events:

- Revolver spin - played at spin start
- Coin collect - played on reward claim
- Item reward - played per reward list item pop-in
- Game Over - played on bomb hit
- Count-up tick - played each increment during reward count-up

7. Quick Start for Developers

Follow these steps to get the project running from scratch:

Step 1 - Open the Project

- Unity version: 2021.3.x LTS
- Open the project in Unity Hub - no additional package installs required

Step 2 - Scene Setup (Fresh)

- Open the main game scene
- If starting from a blank scene: Tools → Wheel of Fortune → Setup Scene Hierarchy
- Do NOT run Setup Scene Hierarchy on an existing, configured scene

Step 3 - Configure Starting State

- Tools → Wheel of Fortune → Player Settings
- Set Starting Coins and Starting Money to desired test values
- Click Apply to PlayerPrefs

Step 4 - Inspect / Edit Slice Data

- Tools → Wheel of Fortune → View Slice Datas to browse all 42 Bronze entries
- Use Edit to modify an existing slice or Select to navigate to the asset
- Tools → Wheel of Fortune → Slice Data Creator to add new slices

Step 5 - Play

- Hit Play in the Unity Editor - the game starts at zone 1
- Use DemoSettings in the Inspector to enable auto-spin for rapid zone testing

Resetting Between Test Runs

- Tools → Wheel of Fortune → Player Settings → Clear All to wipe PlayerPrefs
- Or Reset to Defaults to restore factory values without a full wipe

8. Gameplay Video & Screenshots

A full gameplay recording is available on YouTube (recorded at 16:9):

[Watch Gameplay on YouTube → https://youtu.be/NrDbk1rdM7E](https://youtu.be/NrDbk1rdM7E)

Screenshots are available in the /Screenshots folder at 20:9, 16:9, and 4:3 resolutions. UI panels covered: Game Over, Reward Complete, and Special Offer.